

# Representing the Batching Constraint in a Heterogeneous Graph for the Flexible Job Shop Problem

Mariana Correia

September 29, 2026

## 1 Base problem and graph

### 1.1 Flexible Job Shop Problem

Let  $\mathcal{J} = \{1, \dots, n\}$  be the set of jobs and  $\mathcal{M} = \{1, \dots, m\}$  the set of machines. Each job  $j \in \mathcal{J}$  is an ordered sequence of  $n_j$  operations  $\mathcal{O}_j = (O_{j,1}, \dots, O_{j,n_j})$ , and  $\mathcal{O} = \bigcup_j \mathcal{O}_j$ . For an operation  $o = O_{j,k}$  define

$$\text{job}(o) = j, \quad \kappa(o) = k, \quad (1)$$

where  $\kappa(o)$  is the *order index* of  $o$  inside its job. Operation  $o$  can be processed on any machine of its eligible set  $\mathcal{M}_o \subseteq \mathcal{M}$ , with processing time  $p_{o,\mu} > 0$  for  $\mu \in \mathcal{M}_o$  (and  $p_{o,\mu} = 0 \Leftrightarrow \mu \notin \mathcal{M}_o$ ).

### 1.2 Heterogeneous graph

Following Echeverria (2025), the state is a heterogeneous graph  $G = (\mathcal{V}, \mathcal{E})$  with  $\mathcal{V} = \mathcal{V}_J \cup \mathcal{V}_O \cup \mathcal{V}_M$  (job, operation and machine nodes) and the relations

- (operation, belongs, job):  $o \rightarrow \text{job}(o)$ ;
- (operation, prec, operation):  $O_{j,k+1} \rightarrow O_{j,k}$ , plus self-loops;
- (operation, exec, machine) and its reverse:  $o \leftrightarrow \mu$  for  $\mu \in \mathcal{M}_o$ , each carrying an attribute vector  $\mathbf{a}_{o\mu} \in \mathbb{R}^5$ ;
- (job, listens, job) and (machine, listens, machine): fully connected.

The node features are  $\mathbf{x}_j \in \mathbb{R}^4$ ,  $\mathbf{x}_o \in \mathbb{R}^2$  and  $\mathbf{x}_\mu \in \mathbb{R}^3$ .

## 2 FJSP with batching

### 2.1 Families

Let  $\mathcal{F} = \{1, \dots, |\mathcal{F}|\}$  be a set of *families* and  $\varphi : \mathcal{O} \rightarrow \mathcal{F}$  the *family map*. Operations of the same family have similar characteristics and may be processed together in a *batch*. Let

$$\mathcal{O}^f = \{o \in \mathcal{O} : \varphi(o) = f\}. \quad (2)$$

Two rules define which operations may be processed together:

$$|\mathcal{O}^f \cap \mathcal{O}_j| \leq 1 \quad \forall f \in \mathcal{F}, \forall j \in \mathcal{J}, \quad (\text{F1})$$

$$|\kappa(o) - \kappa(o')| \leq \Delta \quad \forall o, o' \text{ in the same batch}, \quad (\text{F2})$$

with  $\Delta = 2$ . Rule (F1) is a property of the instance: a family contains at most one operation from each job. Rule (F2) is the *order difference* constraint and formalises “similar characteristics”.

Table 1: Family parameters.

| Symbol                                                        | Meaning                                                         |
|---------------------------------------------------------------|-----------------------------------------------------------------|
| $B_f \in \mathbb{N}$                                          | maximum number of operations in a batch of family $f$           |
| $\Delta \in \mathbb{N}$                                       | maximum order difference inside a batch ( $\Delta = 2$ )        |
| $\mathcal{M}_f = \bigcap_{o \in \mathcal{O}^f} \mathcal{M}_o$ | machines eligible for family $f$ (may contain several machines) |
| $\bar{p}_{f,\mu} = \max_{o \in \mathcal{O}^f} p_{o,\mu}$      | upper bound on the batch processing time of $f$ on $\mu$        |

Two operations can share a batch only if their positions in their jobs' routings differ by at most  $\Delta$ . A family may contain operations that are further apart than  $\Delta$ , but those operations can never be in the same batch. Rule (F2) is enforced in the model by (B2a). The family defines which operations are candidates for batching and the order difference restricts which candidates are compatible. For this reason the family feature and the batching constraint are introduced together.

## 2.2 Mixed-integer formulation

For each family  $f$ , let  $\mathcal{B}_f = \{1, \dots, |\mathcal{O}^f|\}$  be the index set of *potential batches*, and  $\mathcal{B} = \bigcup_f \mathcal{B}_f$ . Every batch  $b \in \mathcal{B}_f$  has family  $f(b) = f$ . Let  $L$  be a sufficiently large constant, for example  $L = \sum_o \max_{\mu} p_{o,\mu}$ . Batching is *parallel*: the operations of a batch are processed simultaneously on one machine, and the batch lasts as long as its longest operation. No setup time is considered.

### Decision variables.

- $x_{o,b} \in \{0, 1\}$ : operation  $o$  is assigned to batch  $b$ , defined only for  $b \in \mathcal{B}_{\varphi(o)}$ ;
- $u_b \in \{0, 1\}$ : batch  $b$  is used;
- $y_{b,\mu} \in \{0, 1\}$ : batch  $b$  is processed on machine  $\mu \in \mathcal{M}_{f(b)}$ ;
- $z_{b,b'} \in \{0, 1\}$ : batch  $b$  precedes batch  $b'$  if both are on the same machine;
- $S_b, C_b, p_b \geq 0$ : start, completion and processing time of batch  $b$ ;
- $S_o, C_o \geq 0$ : start and completion time of operation  $o$ ;
- $C_{\max} \geq 0$ : makespan.

## Model.

$$\min C_{\max} \quad (3)$$

$$\text{s.t.} \quad \sum_{b \in \mathcal{B}_{\varphi(o)}} x_{o,b} = 1 \quad \forall o \in \mathcal{O} \quad (\text{B1})$$

$$x_{o,b} = 0 \quad \text{if } \varphi(o) \neq f(b) \quad \forall o \in \mathcal{O}, b \in \mathcal{B} \quad (\text{B2})$$

$$x_{o,b} + x_{o',b} \leq 1 \quad \forall b, o, o' \in \mathcal{O}^{f(b)} : |\kappa(o) - \kappa(o')| > \Delta \quad (\text{B2a})$$

$$\sum_{o \in \mathcal{O}^{f(b)}} x_{o,b} \leq B_{f(b)} u_b, \quad x_{o,b} \leq u_b \quad \forall b \in \mathcal{B}, o \in \mathcal{O}^{f(b)} \quad (\text{B3})$$

$$\sum_{\mu \in \mathcal{M}_{f(b)}} y_{b,\mu} = u_b, \quad y_{b,\mu} = 0 \quad \forall \mu \notin \mathcal{M}_{f(b)} \quad \forall b \in \mathcal{B} \quad (\text{B4})$$

$$p_b \geq p_{o,\mu} (x_{o,b} + y_{b,\mu} - 1) \quad \forall b, o \in \mathcal{O}^{f(b)}, \mu \in \mathcal{M}_{f(b)} \quad (\text{B4a})$$

$$C_b = S_b + p_b \quad \forall b \in \mathcal{B} \quad (\text{B5})$$

$$S_b - L(1 - x_{o,b}) \leq S_o \leq S_b + L(1 - x_{o,b}) \quad \forall o, b \in \mathcal{B}_{\varphi(o)} \quad (\text{B5a})$$

$$C_b - L(1 - x_{o,b}) \leq C_o \leq C_b + L(1 - x_{o,b}) \quad \forall o, b \in \mathcal{B}_{\varphi(o)} \quad (\text{B5b})$$

$$S_{O_{j,k+1}} \geq C_{O_{j,k}} \quad \forall j \in \mathcal{J}, k < n_j \quad (\text{B6})$$

$$S_{b'} \geq C_b - L(3 - y_{b,\mu} - y_{b',\mu} - z_{b,b'}) \quad \forall b < b', \mu \in \mathcal{M} \quad (\text{B7})$$

$$S_b \geq C_{b'} - L(2 - y_{b,\mu} - y_{b',\mu} + z_{b,b'}) \quad \forall b < b', \mu \in \mathcal{M} \quad (\text{B7'})$$

$$C_{\max} \geq C_b \quad \forall b \in \mathcal{B}. \quad (4)$$

## Interpretation.

- (B1)–(B2): every operation belongs to exactly one batch, and every batch contains only one family.
- (B2a): order difference. Two operations whose order indices differ by more than  $\Delta = 2$  cannot be in the same batch. Every batch  $b$  therefore satisfies  $\max_{o \in b} \kappa(o) - \min_{o \in b} \kappa(o) \leq \Delta$ .
- (B3): batch capacity. Unused batches are empty.
- (B4): a family may have several eligible machines ( $|\mathcal{M}_f| \geq 1$ ), and every used batch is assigned to exactly one of them. This is the flexibility of the FJSP, lifted to the batch level.
- (B4a): parallel batching. The batch processing time is the longest processing time among its members on the selected machine,  $p_b = \max_{o \in b} p_{o,\mu}$ . Minimising  $C_{\max}$  drives  $p_b$  down to this maximum.
- (B5)–(B5b): all operations of a batch start together and finish together when the longest one finishes.
- (B6): routing precedence inside each job.
- (B7)–(B7'): batches that share a machine do not overlap.

By (F1) and (B2), a batch never contains two operations of the same job, so the simultaneity constraints (B5a)–(B5b) never conflict with the precedence constraint (B6) inside a batch. Without batching ( $B_f = 1$  for all  $f$ ), the model reduces to the classical FJSP.

### 3 Family feature on the existing nodes

The family index is a label that only has meaning inside one instance: family 3 of one instance has nothing in common with family 3 of another. Feeding the index to the network, as a number or as a learned embedding, would therefore carry no transferable information. Which operations share a family is instead encoded *structurally*, by the family node (Representation A) or by the batch edges (Representation B). The existing node features are extended with numeric descriptors of the family:

$$\text{Operation node: } \mathbf{x}'_o = \left[ \mathbf{x}_o \parallel \mathbb{K}[\varphi(o) \neq \emptyset] \parallel \frac{\kappa(o)}{n_{\text{job}(o)}} \right], \quad (5)$$

$$\text{Job node: } \mathbf{x}'_j(t) = \left[ \mathbf{x}_j(t) \parallel \mathbb{K}[\varphi(o_j^t) \neq \emptyset] \parallel B_{\varphi(o_j^t)} \parallel \pi_j(t) \right], \quad (6)$$

where  $\mathbb{K}[\varphi(o) \neq \emptyset]$  indicates that  $o$  belongs to a family,  $o_j^t$  is the next unscheduled operation of job  $j$  at decision step  $t$ ,  $B_{\varphi(o_j^t)}$  is its capacity (1 if it has no family), and

$$\pi_j(t) = \min \left( B_{\varphi(o_j^t)} - 1, |\{j' \neq j : \varphi(o_{j'}^t) = \varphi(o_j^t), |\kappa(o_{j'}^t) - \kappa(o_j^t)| \leq \Delta, \mathcal{M}_{o_{j'}^t} \cap \mathcal{M}_{o_j^t} \neq \emptyset, r_{j'} \leq s_j\}| \right) \quad (7)$$

is the number of other jobs that would join the batch if job  $j$  were dispatched at its earliest start  $s_j$  ( $r_{j'}$  is the ready time of job  $j'$ ). It follows exactly the batch completion rule of Section 7. The operation's family is static. The job's family is dynamic and changes as the job advances through its routing. The normalised order index in (5) gives the network the information it needs to reason about the order difference (F2).

### 4 Representation A: family (batch) node type

A fourth node type is added:

$$\mathcal{V} = \mathcal{V}_J \cup \mathcal{V}_O \cup \mathcal{V}_M \cup \mathcal{V}_F, \quad |\mathcal{V}_F| = |\mathcal{F}|. \quad (8)$$

Node  $f \in \mathcal{V}_F$  represents the open batch of family  $f$ .

**Node features.**

$$\mathbf{x}_f(t) = \left[ B_f \parallel \bar{p}_f \parallel \frac{|\mathcal{M}_f|}{m} \parallel \frac{n_f(t)}{B_f} \parallel \frac{\kappa_f^{\max}(t) - \kappa_f^{\min}(t)}{\Delta} \parallel \frac{r_f(t)}{|\mathcal{O}^f|} \right], \quad (9)$$

with

$$\bar{p}_f = \frac{1}{|\mathcal{M}_f|} \sum_{\mu \in \mathcal{M}_f} \bar{p}_{f,\mu}, \quad R_f(t) = \{o_j^t \in \mathcal{O}^f : r_j \leq t_f\}, \quad n_f(t) = |R_f(t)|, \quad (10)$$

where  $t_f = \min_{j: o_j^t \in \mathcal{O}^f} s_j$  is the earliest time a batch of  $f$  can start, so  $R_f(t)$  holds the members that could share that batch under the completion rule of Section 7, where  $\kappa_f^{\min}(t), \kappa_f^{\max}(t)$  are the smallest and largest order indices in  $R_f(t)$  and  $r_f(t)$  is the number of members not yet scheduled. The components are the maximum number of operations, the batch processing time (the longest processing time, averaged over the eligible machines), the eligible machines, the current fill level, the current order spread and the fraction of the family still to be scheduled. The family is identified by the node itself, so no identifier feature is needed (Section 3). The fill level  $n_f(t)/B_f$  is dynamic, and  $1 - n_f(t)/B_f$  is the remaining capacity. An order spread above 1 means that not all ready operations can share one batch because of (F2).

**New relations.**

$$\mathcal{E}_{OF} = \{(o, \varphi(o)) : o \in \mathcal{O}\} \quad (\text{operation, in\_family, family}) \text{ and reverse has,} \quad (11)$$

$$\mathcal{E}_{FM} = \{(f, \mu) : f \in \mathcal{F}, \mu \in \mathcal{M}_f\} \quad (\text{family, batch\_exec, machine}) \text{ and reverse,} \quad (12)$$

plus a self-loop (family, **self**, family) on every family node, so that a family node keeps its own state after all its operations have been scheduled. When the last member of a family is scheduled, its **batch\_exec** edges are removed. Each  $(f, \mu) \in \mathcal{E}_{FM}$  carries the machine-specific batch information. It uses the same dimension as the existing **exec** edges:

$$\mathbf{a}_{f\mu} = \left[ \bar{p}_{f,\mu}, \frac{\bar{p}_{f,\mu}}{\sum_{\mu' \in \mathcal{M}_f} \bar{p}_{f,\mu'}}, 0, 0, 0 \right] \in \mathbb{R}^5, \quad (13)$$

**Size.**  $|\mathcal{E}_{OF}| = |\mathcal{O}|$  and  $|\mathcal{E}_{FM}| = \sum_f |\mathcal{M}_f| \leq |\mathcal{F}| m$ . The graph grows linearly.

## 5 Representation B: batch edge type

No node type is added. Batch compatibility is encoded directly between operations by a new relation (operation, **batch**, operation):

$$\mathcal{E}_{OO}^{\text{batch}} = \left\{ (o, o') \in \mathcal{O}^2 : \varphi(o) = \varphi(o'), \text{ job}(o) \neq \text{job}(o'), |\kappa(o) - \kappa(o')| \leq \Delta, \mathcal{M}_o \cap \mathcal{M}_{o'} \neq \emptyset \right\}. \quad (14)$$

The relation is symmetric. It has no self-loops, because every operation already has one in the **prec** relation. The condition  $|\kappa(o) - \kappa(o')| \leq \Delta$  applies the order difference (F2) directly in the graph. Operations of the same family that are more than  $\Delta = 2$  positions apart are not connected, so the network never receives a message suggesting an infeasible batch. A batch is feasible under (F2) exactly when its operations form a clique in  $\mathcal{E}_{OO}^{\text{batch}}$ .

**Edge features.**

$$\mathbf{a}_{oo'} = \left[ \frac{|\kappa(o) - \kappa(o')|}{\Delta}, \frac{|\mathcal{M}_o \cap \mathcal{M}_{o'}|}{|\mathcal{M}_o \cup \mathcal{M}_{o'}|}, \frac{1}{|\mathcal{M}_o \cap \mathcal{M}_{o'}|} \sum_{\mu \in \mathcal{M}_o \cap \mathcal{M}_{o'}} \max(p_{o,\mu}, p_{o',\mu}), B_{\varphi(o)}, 0 \right] \in \mathbb{R}^5. \quad (15)$$

The components are the order distance, the machine overlap (Jaccard index), the processing time of the pair if batched in parallel (the longer of the two), and the capacity. A zero is appended so that all attributed edge types have the same dimension. The family has no node of its own in this representation, so its batch-level parameters are also appended to the operation features:

$$\mathbf{x}_o'' = \left[ \mathbf{x}_o' \parallel B_{\varphi(o)} \parallel \bar{p}_{\varphi(o)} \right]. \quad (16)$$

**Size.**  $|\mathcal{E}_{OO}^{\text{batch}}| \leq \sum_f |\mathcal{O}^f| (|\mathcal{O}^f| - 1)$ , which is quadratic in the family size. By (F1),  $|\mathcal{O}^f| \leq n$ , so the relation has at most  $|\mathcal{F}| n^2$  edges.

## 6 Graph attention message passing

Both representations use the same heterogeneous GAT layer. For every relation  $r = (\tau_s, \rho, \tau_d)$  and edge  $(u, v)$  of type  $r$ , the layer computes (GATv2 with edge attributes):

$$e_{uv}^r = \mathbf{a}_r^\top \text{LeakyReLU}(\mathbf{W}_s^r \mathbf{h}_u + \mathbf{W}_d^r \mathbf{h}_v + \mathbf{W}_e^r \mathbf{a}_{uv}), \quad (17)$$

$$\alpha_{uv}^r = \frac{\exp(e_{uv}^r)}{\sum_{w \in \mathcal{N}_r(v)} \exp(e_{uw}^r)}, \quad (18)$$

$$\mathbf{h}_v^r = \sum_{u \in \mathcal{N}_r(v)} \alpha_{uv}^r \mathbf{W}_s^r \mathbf{h}_u, \quad (19)$$

$$\mathbf{h}_v^{(l+1)} = \sigma \left( \text{AGG}_{r: \tau_d(r)=\text{type}(v)} \mathbf{h}_v^r \right), \quad (20)$$

where the term  $\mathbf{W}_e^r \mathbf{a}_{uv}$  is omitted for relations without edge attributes, AGG is a sum or mean over relations, and multi-head outputs are concatenated.

- **Representation A:** information reaches a machine from an operation in two hops (operation  $\rightarrow$  family  $\rightarrow$  machine). The attention  $\alpha_{of}^{\text{has}}$  lets the family node weigh its candidate members, and machines receive batch-level times through  $\mathbf{a}_{f\mu}$ .
- **Representation B:** batching information is exchanged in one hop (operation  $\leftrightarrow$  operation). The attention coefficient  $\alpha_{oo'}^{\text{batch}}$  directly scores how suitable it is to batch  $o$  with  $o'$ .

## 7 Consequences for the decision process

Both representations keep the action space of the operation–job–machine graph. The policy scores the (machine, exec, job) edges and selects one pair  $a_t = (\mu, j)$ . The two representations therefore differ *only* in the graph given to the network, which makes their comparison fair and lets the same agent be used for both.

The selected pair opens a batch  $b_t$  for the current operation  $o = o_j^t$  on machine  $\mu$ , starting at

$$S_{b_t} = \max(\text{free time of } \mu, \text{ ready time of } j). \quad (21)$$

If  $o$  belongs to a family  $f$ , the environment completes the batch with other jobs  $j'$  whose current operation  $o' = o_j^t$ , satisfies

$$\varphi(o') = f, \quad \mu \in \mathcal{M}_{o'}, \quad \text{ready time of } j' \leq S_{b_t}, \quad (22)$$

adding them in order of earliest ready time while

$$|b_t| \leq B_f, \quad \max_{o \in b_t} \kappa(o) - \min_{o \in b_t} \kappa(o) \leq \Delta. \quad (23)$$

A candidate joins only if it is already ready, so joining never delays the batch. The batch then occupies  $\mu$  during  $[S_{b_t}, S_{b_t} + \max_{o \in b_t} p_{o,\mu}]$  (parallel batching), and all of its jobs advance to their next operation.

**Action-edge features.** After every decision each (machine, exec, job) edge  $(\mu, j)$  is rebuilt with

$$\mathbf{a}_{\mu j} = [p_b, \frac{p_{o_j^t, \mu}}{\sum_{\mu'} p_{o_j^t, \mu'}}, s_{j\mu}, s_{j\mu} + p_b, s_{j\mu} - \text{free time of } \mu], \quad (24)$$

where  $s_{j\mu}$  is the earliest start of  $o_j^t$  on  $\mu$ . In Representations A and B,  $p_b$  is the processing time of the batch the action would dispatch (its longest member), i.e. what the action actually costs. The baseline uses the individual time  $p_{o_j^t, \mu}$ , so its graph carries no batching information.

**Warm start.** The mask and the dispatch rule used as the warm-start teacher are those of the operation–job–machine graph: earliest start time, with ties broken by the earliest completion time. Both quantities are visible to the network as columns of  $\mathbf{a}_{\mu j}$ , so the teacher can actually be imitated.

## 8 Comparison

To measure what each representation contributes, both are compared with a *baseline*. The baseline solves the same batching problem with the same action space and batch completion rule (Section 7), but on the unchanged operation–job–machine graph of Section 1. It has no family node, no batch edges and none of the family features of Section 3, so the network cannot see which operations can be batched. Any improvement of Representation A or B over the baseline can therefore be attributed to how the batching information is represented.

Table 2: Comparison of the two representations.

|                               | A: family node                                                 | B: batch edge                                                      |
|-------------------------------|----------------------------------------------------------------|--------------------------------------------------------------------|
| New types                     | 1 node type, 2 relations (+ reverses)                          | 1 relation                                                         |
| Number of edges               | $\mathcal{O}( \mathcal{O}  + \sum_f  \mathcal{M}_f )$ (linear) | $\mathcal{O}(\sum_f  \mathcal{O}^f ^2)$ (quadratic in family size) |
| Order difference $\Delta$     | visible through the order spread feature                       | built into the edge set                                            |
| Batch parameters              | stored once on the family node and its edges                   | duplicated on operation nodes and edges                            |
| Hops op $\rightarrow$ machine | 2 (via family)                                                 | 1 (via <code>exec</code> )                                         |
| Action                        | (machine, job) + batch completion rule                         | (machine, job) + batch completion rule                             |

## 9 Illustrative example

Consider  $n = 3$  jobs with four operations each,  $m = 2$  machines,  $B_1 = B_2 = 2$  and  $\Delta = 2$ . Two families are shown:

$$\mathcal{O}^1 = \{O_{1,1}, O_{2,2}, O_{3,4}\}, \quad \mathcal{O}^2 = \{O_{1,3}, O_{2,3}\}.$$

The remaining operations are singleton families. Assume all operations in these families are eligible on both machines, and let  $p_{O_{1,1},1} = 5$  and  $p_{O_{2,2},1} = 8$ .

- (F1) holds: each family has at most one operation per job.
- Order difference in  $\mathcal{O}^1$ :  $|1 - 2| = 1 \leq 2$  and  $|2 - 4| = 2 \leq 2$  are compatible, but  $|1 - 4| = 3 > 2$ , so  $O_{1,1}$  and  $O_{3,4}$  can never share a batch. By (B2a), the feasible batches of size two are  $\{O_{1,1}, O_{2,2}\}$  and  $\{O_{2,2}, O_{3,4}\}$ .
- In  $\mathcal{O}^2$ ,  $|3 - 3| = 0$ , so  $\{O_{1,3}, O_{2,3}\}$  is feasible.
- Parallel batching: if  $\{O_{1,1}, O_{2,2}\}$  runs on machine 1, then  $p_b = \max(5, 8) = 8$  and both operations finish at  $S_b + 8$ .
- Representation A adds two family nodes and  $|\mathcal{E}_{OF}| = 5$  edges, one per operation. It also adds edges to the two machines from each family.
- Representation B adds the directed edges  $O_{1,1} \leftrightarrow O_{2,2}$  and  $O_{2,2} \leftrightarrow O_{3,4}$  in family 1 (not  $O_{1,1} \leftrightarrow O_{3,4}$ ), plus  $O_{1,3} \leftrightarrow O_{2,3}$  in family 2. That is 6 edges, excluding self-loops.

## 10 Diagnostic of the batch process and version 2

### 10.1 Diagnostic

The decision process of Section 7 was checked on the 40 instances of the batching dataset, with the configuration used for training (`sel_k` = 1, mask on the earliest completion), replaying the warm-start teacher and a random policy (about 3500 decisions each).

#### Correct behaviour.

- Every batch satisfies the rules: one family, at most  $B_f$  members, order difference  $\leq \Delta$ , the machine eligible for every member, every member ready by the start, one operation per job. All schedules pass the independent checker.
- The start time and batch processing time shown on the chosen action edge are exactly those the environment then schedules (all 3515 decisions, both representations).
- The partner count  $\pi_j(t)$  on the job node equals the number of partners the batch rule actually adds in 99.9% of the decisions.
- A partner can lengthen the batch (54% of the batches, +15% of the chosen job’s time on average), since the batch lasts as long as its longest member. This is not a defect: only adding partners that do not lengthen it raised the teacher’s gap to CP-SAT from 0.458 to 0.661 and removed almost all batching. The action edge already shows the batch time, so the policy sees the cost.

**Limitation 1: the batch never waits.** A partner joins only if it is already ready at the start. The teacher missed 214 compatible partners that became ready shortly afterwards (median 0.44 batch times later) and batched 11.7% of the operations, against 18% in the CP-SAT solutions. Letting the batch wait for partners ready within half the chosen job’s time improved the teacher’s mean gap from 0.458 to 0.424, but it helped 18 instances and hurt 17: waiting should be a decision, not a fixed rule.

**Limitation 2: the mask ignores batching.** The mask ranks each job’s machines by the job’s *individual* completion. With `sel_k` = 1, in 14% of the family decisions a larger batch was possible on another machine, and that machine was always masked. The action-edge features and the teacher, however, use the completion of the batch.

### 10.2 Version 2

Version 2 (representations `ojmb_node_v2`, `ojmb_edge_v2`, `ojmb_base_v2`) changes only the action space; the graphs, features and the batch rule are those of Sections 3–5.

**Wait actions.** For an allowed action  $(\mu, j)$  whose current operation belongs to a family, let  $b$  be the batch it dispatches now at  $s_{j\mu}$ , and  $s_{j\mu}^w$  the ready time of the next compatible partner: the smallest  $r_{j'} > s_{j\mu}$  over the jobs  $j' \notin b$  whose current operation is in the same family, is eligible on  $\mu$  and keeps the order difference of  $b \cup \{o_{j'}^t\}$  within  $\Delta$  (defined when  $|b| < B_f$ ). If the batch dispatched at  $s_{j\mu}^w$  is larger than  $b$ , a second action edge  $(\mu, j)$  is added, “wait for the next partner”: it starts at  $s_{j\mu}^w$  and applies the same batch rule there. Its features are those of the immediate action with the later start, the batch time of the larger batch (the individual time for the baseline) and the machine idle time  $s_{j\mu}^w - F_\mu$  that waiting leaves. The policy therefore sees both options and their costs, and chooses.



**Batch-aware mask.** Each job keeps the `sel_k` machines with the earliest completion of the batch the action dispatches,  $s_{j\mu} + p_b$ , instead of  $s_{j\mu} + p_{o_j^t, \mu}$ . A wait action is allowed exactly when its immediate twin is.

**Effect on the teacher and on a random policy.** A wait option exists in 43% of the teacher’s decisions. The teacher, which minimises the chosen job’s completion, never takes one, and its gap changes from 0.458 to 0.466 (family node) and from 0.438 to 0.460 (baseline, whose teacher ranks by individual times while the mask now ranks by batch times). A random policy batches more with the wait actions (20.8%  $\rightarrow$  23.9% of the operations). Whether waiting pays off must therefore be learned by BOPO beyond the teacher; the comparison with version 1 is made on trained policies.

**Verification.** The version-1 environments produce exactly the same schedules before and after the change (12 configurations: three representations, `sel_k`  $\in \{1, 100\}$ , teacher and random policy). `tests/test_batching_v2.py` checks that the version-2 schedules are feasible, that every wait action is the twin of an allowed action, starts later, shows its start, and when taken schedules the larger batch at exactly that start; that the mask keeps the machine with the earliest batch completion; and that the three representations train.

### 10.3 Batch size on the action edge

In version 2 the immediate action and its wait twin differ mainly in how many operations they dispatch, which none of the five action-edge columns shows (a batch whose partners are short has the same start and batch time as the job alone). A sixth column is therefore added to the action edge of every version-2 representation:

$$\mathbf{a}_{\mu j} = \left[ p_b, \frac{p_{o_j^t, \mu}}{\sum_{\mu'} p_{o_j^t, \mu'}}, s_{j\mu}, s_{j\mu} + p_b, s_{j\mu} - F_\mu, n_b \right] \in \mathbb{R}^6, \quad (25)$$

where  $n_b = |b|$  is the number of operations of the batch the action dispatches (for a wait action, the larger batch). The baseline receives 0 in this column, so it stays blind to batching. All attributed relations are zero-padded to dimension 6, and the edge dimension of the GNN is read from the environment (the version-1 environments keep 5, so their trained models are unchanged). Adding the column to every representation, not only to the new one below, keeps the information of the action edges equal across representations.

## 11 Representation C: batching as feature constraints

Representation C (`ojmb_feat_v2`) adds no node type and no edge type: its graph has exactly the node and edge types of the baseline, and the batching constraint is expressed only through features of existing nodes and edges.

**Why not the family number.** The family index only has meaning inside one instance (Section 3). Moreover, operations of the same family belong to different jobs and are never adjacent in the operation–job–machine graph; they only meet through a machine node, whose aggregation mixes every family eligible on it. Equality of a family number would therefore be nearly invisible to the network. What the policy needs from “same family” is who can be batched, how many partners there are and what the batch costs; these are computed by the environment and given as numbers.

**Features.** Let  $P(o) = \{o' \in \mathcal{O}^{\varphi(o)} : \text{job}(o') \neq \text{job}(o), |\kappa(o') - \kappa(o)| \leq \Delta\}$  be the compatible partners of  $o$ , and  $U(t)$  the operations not yet scheduled at step  $t$ .

$$\text{Operation: } \mathbf{x}_o^C(t) = \left[ \mathbf{x}_o \parallel \mathbb{I}[\varphi(o) \neq \emptyset] \parallel \frac{\kappa(o)}{n_{\text{job}(o)}} \parallel B_{\varphi(o)} \parallel \bar{p}_{\varphi(o)} \parallel c_o(t) \parallel \frac{r_{\varphi(o)}(t)}{|\mathcal{O}^{\varphi(o)}|} \right], \quad (26)$$

$$c_o(t) = |\{o' \in P(o) \cap U(t) : \mathcal{M}_o \cap \mathcal{M}_{o'} \neq \emptyset\}|, \quad (27)$$

$$\text{Job: } \mathbf{x}_j^C(t) = \mathbf{x}'_j(t) \quad (\text{as in A and B: flag, } B_f, \pi_j(t)), \quad (28)$$

$$\text{Machine: } \mathbf{x}_\mu^C(t) = \left[ \mathbf{x}_\mu \parallel \max_{j: \varphi(o_j^t) \neq \emptyset, \mu \in \mathcal{M}_{o_j^t}} \frac{|b(j, \mu)|}{B_{\varphi(o_j^t)}} \right], \quad (29)$$

$$\text{Exec edge } (o, \mu) : a_{o\mu,3}(t) = |\{o' \in P(o) \cap U(t) : \mu \in \mathcal{M}_{o'}\}|, \quad (30)$$

where  $b(j, \mu)$  is the batch the action  $(\mu, j)$  would dispatch now and column 3 of the exec edge was always zero before. The action edge is that of version 2, including  $n_b$ .

**The batching action with features only.** The action is the existing (machine, exec, job) edge. Choosing it dispatches a batch, and its features describe that batch completely:  $n_b$  operations start at  $s_{j\mu}$ , last  $p_b$  and leave the machine idle for  $s_{j\mu} - F_\mu$ . The wait twin of version 2 is another edge of the same type, so waiting is represented by features too.

**What the comparison measures.** C lies between the baseline (no batching information) and A and B (the same kind of information plus structure). Its comparison with them is read on every group of metrics: schedule quality (whether features alone suffice), constraint handling (a check), expressiveness (structure gain is expected close to the baseline's, since no edges are added, while distinct inputs rise with the richer features), heterophily, batching behaviour (whether C uses batching as much as A and B), efficiency (C adds no nodes or edges, so it should be the cheapest batching representation) and the per-step training curves (whether features give the same early advantage as structure).